

SetuCredit

RBI ULI Integration Guide

Step-by-step: RBIH onboarding + code integration

Companion scaffold: `backend/app/dpi/uli.py` + `registry.py`

Sources: docs.rbihub.in (RBIH ULI docs, 2026)

Step 0 - Decide your role (day 1)

SetuCredit is middleware, not the lender, so you have two paths. Path A (recommended for the hackathon-to-pilot jump): ride the partner NBFC's existing ULI access. You send the signed appraisal package over the HMAC-signed NBFC webhook you already have, and the NBFC pulls ULI itself. Zero RBIH paperwork for you. Path B: onboard as a lender yourselves and call ULI directly. Everything below describes Path B; the code scaffold supports both because stubs stay default until credentials land.

Step 1 - RBIH onboarding application (weeks 1-2)

- Apply on the RBIH ULI onboarding page as a lender: company details, designated contacts, and service requirements.
- Review takes about 1-2 weeks. On approval you are registered in RBIH's Fintech Repository and receive production keys later.
- Keep a single onboarding owner + SPOC thread; every later step (whitelisting, subscriptions, go-live) goes through them.

Step 2 - Document package (parallel with Step 1)

- DOC 01: architecture diagram + data-flow (reuse docs/ARCHITECTURE.md; HLD wins on conflicts).
- DOC 02: anti-malware / endpoint policy.
- DOC 03: VAPT report from a CERT-IN empanelled auditor, less than 6 months old at submission (valid 1 year once accepted).
- DPDP consent artefacts: consent request format, time-bound + revocable proof your app/consent/service.py already enforces (403 gate).

Step 3 - IP whitelisting + UAT credentials (days)

- Email your static egress IPs to support@rbihub.io. Review: 2-3 business days.
- UAT credentials arrive ~1 business day later by secure one-to-one email to the registered contact. Check spam, then confirm with your SPOC.
- Ops note: whitelisted IPs must be static (NAT gateway / egress proxy). Ephemeral cloud IPs will break production auth.

Step 4 - Developer portal + sandbox keys

- Sign in at <https://am.nonprod.rbihub.io/devportal> with UAT credentials.
- Applications tab > your lender workspace > Sandbox Keys: copy the Bearer token (scoped, expiring; regenerate as needed).
- Subscriptions tab > + Subscribe APIs: subscribe to the lending-history / credit APIs you need (full list at docs.rbihub.in).
- Activation is manual: email lender-onboarding@rbihub.in with subject 'API Subscription Approval Request - [Org]' listing the APIs. Subscriptions do nothing until RBIH activates them.

Step 5 - Code integration (this repo)

The scaffold is already in place and inert by default:

```
backend/app/dpi/uli.py          UliApiAdapter (OAuth2 + HMAC + mTLS)
backend/app/dpi/registry.py     stubs default; live ULI iff
                                ULI_MODE=live + credentials present
backend/app/config.py          ULI_* settings (non-prod URLs default)
backend/tests/test_dpi_registry.py 4 routing/shape tests
```

5a. Configure (never commit secrets)

```
ULI_MODE=live
ULI_BASE_URL=https://extgw.nonprod.rbihub.io
ULI_TOKEN_URL=https://auth.nonprod.rbihub.io/oauth/token
ULI_CLIENT_ID=<from portal>
ULI_CLIENT_SECRET=<from portal>
# mTLS (if your subscription requires it):
# ULI_MTLS_CERT=/run/secrets/uli-client.crt
# ULI_MTLS_KEY=/run/secrets/uli-client.key
```

5b. Finalize the three TODO(ULI) points in uli.py

- Token path/grant at the RBIH auth service for your subscription.
- Lending-history API path replacing POST /uli/lending-history.
- HMAC canonical string + header names from the RBIH signing guide (scaffold signs the raw JSON body with HMAC-SHA256).

5c. Verify locally

```
docker compose -f infra/docker-compose.yml up -d postgres redis
uv run pytest backend/tests/test_dpi_registry.py -q    # 4 passed
uv run pytest -q                                       # full suite
```

Step 6 - Consent + pass-through compliance

- Every ULI call must carry borrower consent proof: the scaffold sends `consent_ref` with each pull; the consent 403 gate stays upstream of the fan-out.
- Pass-through by design: `UliApiAdapter` maps the gateway response to exactly `{existing_loans, repayment_regular_months, delinquencies_12m}` and returns it in-memory. No payload logging, no new tables (see test asserting the shape).
- Orchestrator `run.py` already bounds each source by `dpi_timeout_seconds` and isolates failures: a ULI error/timeout degrades that source, never the whole pull.

Step 7 - Sandbox testing (1-2 weeks)

- Drive the full flow against non-prod: `session > OTP > consent > appraise`, with `uli` in scope; record sandbox responses as fixtures.
- Add contract tests with recorded fixtures (`MockTransport` pattern in `test_dpi_registry.py`); assert the mapped shape, never raw payloads.
- Load-check the fan-out: ULI p95 must sit inside `dpi_timeout_seconds`, else raise the timeout or narrow the subscribed APIs.

Step 8 - Go-live

- Execute the MSA with RBIH, receive production keys, rotate all sandbox credentials out of every environment.
- Point `ULI_BASE_URL/ULI_TOKEN_URL` at production, mount prod mTLS certs from the secret store, confirm egress IPs still whitelisted.
- ULI bills per API call: the mapped adapter is the metering point - log one usage line per fetch (`source_id`, `ok`, `latency`, `record_count`) for invoice reconciliation. No payload contents in usage logs.

Step 9 - Operate

- Rotate client secrets + mTLS certs on schedule; keep VAPT validity unbroken (re-audit yearly).
- Monitor per-source `ok-rate/latency` from `run_pull` results; alert on ULI timeout spikes before lenders notice.
- Keep stubs working: `ULI_MODE=stub` must always boot with no credentials so dev/CI never depend on the gateway.